

need not retrieve it from the main memory. This process is known as cache-snooping.

With the theory out of our way, how can we check the cache? Here's a simple command that does that job:

```
# getconf -a | grep -i cache
```

The following is the output from my test system:

```
LEVEL1_ICACHE_SIZE      32768
LEVEL1_ICACHE_ASSOC      8
LEVEL1_ICACHE_LINESIZE  64
LEVEL1_DCACHE_SIZE      32768
LEVEL1_DCACHE_ASSOC      8
LEVEL1_DCACHE_LINESIZE  64
LEVEL2_CACHE_SIZE      2097152
LEVEL2_CACHE_ASSOC      8
LEVEL2_CACHE_LINESIZE  64
LEVEL3_CACHE_SIZE      0
LEVEL3_CACHE_ASSOC      0
LEVEL3_CACHE_LINESIZE  0
LEVEL4_CACHE_SIZE      0
LEVEL4_CACHE_ASSOC      0
LEVEL4_CACHE_LINESIZE  0
```

You can also use the *x86info* tool (you need to install it first). It gives very detailed information about your CPU and types of cache. Figure 4 shows the output on my system when I execute the *x86info -c* command.

You can also check */var/log/dmesg* for this information.

Locality of reference

Applications normally tend to behave in a particular way when accessing data. An application accessing memory location *X* is more likely to access memory location *X+1* in the next few cycles of execution. This behaviour is known as spatial locality of reference.

Programs that access the memory sequentially will generally get more benefits from the cache. Programs that result in more cache-misses are more expensive, as they increase latency. There is a tool called Valgrind that you can use to profile cache usage. Given a choice, this can help you determine the more efficient program.

```
=6941== cachegrind, a cache and branch-prediction profiler.
=6941== Copyright (C) 2002-2008, and GNU GPL'd, by Nicholas Nethercote et al.
=6941== Using LibVEX rev 1884, a library for dynamic binary translation.
=6941== Copyright (C) 2004-2008, and GNU GPL'd, by OpenWorks LLP.
=6941== Using valgrind-3.4.1, a dynamic binary instrumentation framework.
=6941== Copyright (C) 2000-2008, and GNU GPL'd, by Julian Seward et al.
=6941== For more details, rerun with: -v
=6941==
tarting
inished
=6941==
=6941== I refs: 6,188,126,666
=6941== I1 misses: 666
=6941== L2i misses: 664
=6941== I1 miss rate: 0.00%
=6941== L2i miss rate: 0.00%
=6941==
=6941== D refs: 3,937,849,712 (3,375,259,224 rd + 562,590,488 wr)
=6941== D1 misses: 35,157,103 ( 663 rd + 35,156,440 wr)
=6941== L2d misses: 35,157,720 ( 621 rd + 35,156,435 wr)
=6941== D1 miss rate: 0.8%
=6941== L2d miss rate: 0.8%
=6941==
=6941== L2 refs: 35,157,769 ( 1,329 rd + 35,156,440 wr)
=6941== L2 misses: 35,157,720 ( 1,285 rd + 35,156,435 wr)
=6941== L2 miss rate: 0.3%
root@legacy ~]#
```

Figure 5: Testing application performance using Valgrind

Here are the steps on how to get started with it:

Step 1 Use the *x86info -c* command to get the values of your cache. Valgrind is more interested in: Instruction Cache (I1), Data Level 1 (D1) and Data Level 2 (L2) cache values.

Remember that Valgrind demands the I1 cache value in bytes rounded off to the nearest power of 2, besides the D1 and L2 cache (also in bytes). We also need to jot down the values of associativity and line-size. From my output of *x86info -c*, as shown in Figure 4, I get the following values.

- I1 = 32768,64
- D1 = 32768,8,64
- L2 = 2097152,8,64

Step 2 Now, if I have two applications that perform the same task, I need to check which one performs better with respect to cache-hits and cache-misses. I can use Valgrind to find out:

```
# valgrind -tool=cachegrind -i=32768,8,64 \
--D1=32768,8,64 \
--L2=2097152,8,64 sample1
```

Here I'm testing a sample

application called sample1. Figure 5 shows the output (truncated) of the above command.

As you can see, the cache-miss rate for the application sample1 is very low (below 1 per cent). This means that sample1 makes good use of cache-hits, thereby enhancing the performance. It is always recommended to use Valgrind for new applications before putting it in a commercial environment.

As a general rule, the less the miss-rate, the better!

Here I conclude for this month. Next month will be dedicated to how we can tune memory address and allocation using PAM, TLB, pflush, etc, as well as tune our network performance. **END**

By: Alok Srivastava

The author is the founder of Network NUTS and holds MCP, MCSE, MCDBA, MCT, CCNA, CCNP, RHCE and RHCSS certifications. Under his leadership, Network NUTS has been a winner of the "Best Red Hat Training Partner in North India" for the last three years in a row. He has also been a trainer for the Indian Air Force, NIC, LIC, IFFCO, Wartsila India Ltd, the Government of Rajasthan, Bajaj Allianz, etc. You can reach him at alok@networknuts.net.